

Roadmap Sprint 2 ES

Roadmap Estratégico e Guia de BDD: Sprint 2 — Projeto CAMAAR

Informações Gerais e Governança

- **Prazo de Entrega Final:** 16/06/2026
 - **Product Owner (PO):** Leandro
 - **Scrum Master (SM):** Hugo
-

Estratégia de Desenvolvimento e Consulta Local

Para blindar a avaliação do grupo e seguir rigorosamente as diretrizes pedagógicas da disciplina, adotaremos a seguinte estratégia:

1. **Ambiente Local de Consulta:** Cada membro manterá uma cópia do projeto adiantado salva **apenas localmente** em sua máquina para ser utilizada estritamente como material de referência técnica e consulta de engenharia reversa.
-

Engenharia Reversa do BDD: Como Construir um Histórico de Commits Rico

Como os arquivos `.feature` já foram criados e integrados na Sprint 1, a estrutura de pastas do Cucumber já está pronta no repositório principal. Para demonstrar a aplicação real do ciclo **Red-Green-Refactor** e garantir uma pontuação excelente em boas práticas de engenharia de software, cada membro fará commits incrementais simulando o avanço do desenvolvimento guiado por testes.

Para **cada uma** de suas issues, você deve realizar obrigatoriamente um ciclo de **pelo menos 3 a 4 commits atômicos** seguindo o padrão de mensagens estabelecido abaixo.

Padrão de Commits (`iuricode/padrees-de-commits`)

Utilizaremos os prefixos semânticos baseados nas melhores práticas do mercado:

- `feat` : Nova funcionalidade ou lógica de negócio.
- `test` : Criação, alteração ou execução de testes (RSpec ou Cucumber).
- `refactor` : Modificação no código que não altera o comportamento final (melhoria de arquitetura, limpeza).

- `style` : Mudanças estéticas, formatação de código, CSS ou indentação que não alteram a lógica.
- `docs` : Atualizações na Wiki do repositório ou documentações internas.

O Fluxo de Commits por Issue:

1. Passo 1: Vincular o Teste de Aceitação existente (Estado: RED)

- **O que fazer:** Vá até o arquivo `.feature` da sua issue (que já está no repositório) e configure os `step_definitions` correspondentes com a sintaxe básica em Ruby, mas sem a implementação interna (deixando-os vazios ou falhando com um expect que quebre). Execute o Cucumber localmente e veja o teste falhar.
- **Commit Semântico:** `test: executa cenarios de aceitacao do cucumber para a issue #ISSUE`

2. Passo 2: Construir as Especificações Unitárias/Integração (Estado: RED)

- **O que fazer:** Crie os arquivos de teste RSpec (`spec/models/...` ou `spec/requests/...`) para a funcionalidade correspondente, baseando-se no seu projeto de consulta local. Execute o RSpec e garanta que ele falhe.
- **Commit Semântico:** `test: adiciona especificacoes do rspec em estado falho para a issue #ISSUE`

3. Passo 3: Trazer a Implementação e Validar os Testes (Estado: GREEN)

- **O que fazer:** Copie os arquivos de código real do seu projeto local (Models, Views, Controllers, Migrations ou Rotas) para o repositório oficial da sua branch. Execute o Cucumber e o RSpec. Ambos devem passar com sucesso.
- **Commit Semântico:** `feat: implementa a funcionalidade completa da issue #ISSUE`

4. Passo 4: Polimento, Design e Refatoração (Estado: REFACTOR / STYLE)

- **O que fazer:** Melhore a legibilidade do código, adicione classes de estilo, remova comentários ou ajuste a semântica das tags HTML.
- **Commit Semântico:** `refactor: otimiza consultas e limpa o escopo do controller da issue #ISSUE` ou `style: ajusta o layout das views da issue #ISSUE`

Divisão de Issues e Escopo Detalhado por Integrante

Alan — Módulo: Gerenciamento de Templates de Formulário

Responsabilidade: Cuidar da arquitetura dos moldes das avaliações (estruturas reutilizáveis).

- **#102 — Criar template de formulário**
 - **Ator:** Administrador

- **Funcionalidade:** O Administrador deve conseguir criar um template de formulário contendo perguntas dinâmicas (como perguntas textuais ou do tipo escala Likert de 1 a 5). Isso servirá de molde para gerar as avaliações reais das turmas.
 - **#111 — Visualização dos templates criados**
 - **Ator:** Administrador
 - **Funcionalidade:** Exibir uma lista centralizada contendo todos os templates criados no sistema, permitindo que o administrador saiba quais moldes estão disponíveis para uso.
 - **#112 — Edição e deleção de templates**
 - **Ator:** Administrador
 - **Funcionalidade:** Permitir que o administrador edite o título, descrição ou perguntas de um template, ou faça a exclusão física do mesmo. A regra de negócio crucial é que essa alteração **não deve afetar ou corromper os formulários de avaliação que já foram criados e publicados** anteriormente a partir desse template.
-

Hugo — Módulo: Ciclo de Vida e Preenchimento de Avaliações

Responsabilidade: Cuidar da publicação dos formulários para as turmas e da coleta das respostas dos discentes.

- **#99 — Responder formulário**
 - **Ator:** Participante de uma turma
 - **Funcionalidade:** Permitir que o aluno acesse uma avaliação pendente e preencha as respostas para as perguntas textuais e de múltipla escolha (Likert), salvando o registro com o timestamp atual. Deve haver uma restrição (índice composto exclusivo) impedindo que o mesmo aluno envie mais de uma resposta para o mesmo formulário.
- **#103 — Criar formulário de avaliação**
 - **Ator:** Administrador
 - **Funcionalidade:** O Administrador cria uma avaliação real ligando um dos templates (do Alan) a uma ou mais turmas específicas do semestre letivo. O formulário deve ter data de início, data limite de preenchimento e validações cronológicas (a data limite não pode ser anterior ao início).
- **#109 — Visualização de formulários para responder**
 - **Ator:** Participante de uma turma
 - **Funcionalidade:** Apresentar um painel de pendências para o aluno logado, listando exclusivamente os formulários ativos de turmas em que ele está matriculado e que ainda não foram respondidos por ele. Formulários fora do prazo de validade não devem aparecer.
- **#110 — Visualização de resultados dos formulários**

- **Ator:** Administrador
 - **Funcionalidade:** Permitir que o administrador visualize uma listagem com os formulários criados no sistema para acompanhar quais possuem respostas enviadas e acessar o painel interno de métricas de cada um.
-

Leandro — Módulo: Consolidação de Dados e Permissões Administrativas

Responsabilidade: Extração das métricas do sistema e segmentação das visões gerenciais.

- **#101 — Gerar relatório do administrador**
 - **Ator:** Administrador
 - **Funcionalidade:** Disponibilizar a exportação de dados em formato de arquivo CSV contendo os resultados compilados de um formulário específico (respostas textuais e numéricas). Se o formulário não tiver respostas, o download não deve ser iniciado e um aviso deve ser exibido.
 - **#106 — Sistema de gerenciamento por departamento ★ Bônus**
 - **Ator:** Administrador
 - **Funcionalidade:** O sistema deve filtrar o escopo de atuação do Administrador com base no seu vínculo institucional. Um administrador vinculado ao Departamento de Ciência da Computação (CIC), por exemplo, só poderá gerenciar e ver as turmas e relatórios do CIC, bloqueando o acesso e a interferência em dados de outros departamentos.
 - **#113 — Criação de formulário para docentes ou discentes ★ Bônus**
 - **Ator:** Administrador
 - **Funcionalidade:** Ao criar uma avaliação, o administrador deve poder determinar o público-alvo específico: se o formulário é direcionado apenas para os discentes avaliarem a turma ou apenas para os docentes. O sistema deve garantir que o formulário apareça apenas no painel do público escolhido.
-

Luan — Módulo: Autenticação, Controle de Acesso e Segurança

Responsabilidade: Portão de entrada do sistema e fluxos de gerenciamento de credenciais de usuários.

- **#104 — Sistema de Login**
 - **Ator:** Usuário do sistema
 - **Funcionalidade:** Permitir a autenticação segura no sistema via e-mail ou número de matrícula e senha. Possui uma regra de interface vital: se o perfil logado for um

Administrador, a barra ou menu lateral de navegação deve exibir dinamicamente a opção "Gerenciamento".

- **#105 — Sistema de definição de senha**

- **Ator:** Usuário
- **Funcionalidade:** Quando novos usuários são importados do SIGAA, eles entram no sistema com status "inativo" e sem senha definida. O sistema dispara um e-mail com um token de definição de senha válido por 24 horas. Ao clicar no link e cadastrar uma senha válida, o status do usuário muda para "ativo" e o login é liberado.

- **#107 — Redefinição de senha ★ Bônus**

- **Ator:** Usuário
- **Funcionalidade:** O fluxo tradicional de "Esqueci minha senha". O usuário solicita a troca, recebe um e-mail com token de segurança e consegue atualizar suas credenciais para recuperar o acesso à plataforma.

Ricardo — Módulo: Sincronização e Governança com o SIGAA

Responsabilidade: Camada de persistência, alimentação e consistência histórica dos dados acadêmicos.

- **#98 — Importar dados do SIGAA**

- **Ator:** Administrador
- **Funcionalidade:** Desenvolver o console de integração capaz de ler os arquivos JSON presentes no repositório representando os dados do SIGAA, gerando as instâncias correspondentes de departamentos, cursos, disciplinas e turmas que ainda não existem no banco de dados.

- **#100 — Cadastrar usuários do sistema**

- **Ator:** Administrador
- **Funcionalidade:** Atrelado à rotina de importação, o sistema deve varrer a lista de participantes do SIGAA e criar os registros dos novos usuários na tabela correspondente para que estes possam posteriormente realizar o fluxo de ativação de conta.

- **#108 — Atualizar base de dados com os dados do SIGAA**

- **Ator:** Administrador
- **Funcionalidade:** Rotina periódica de sincronização para manter o sistema atualizado. Deve tratar com segurança eventos como trancamentos de disciplinas e novas matrículas tardias enviadas pelo SIGAA. **Regra de ouro:** se um aluno for desvinculado de uma turma pelo SIGAA, o vínculo dele na turma é inativado, mas o histórico de respostas de avaliações que ele já enviou anteriormente deve ser mantido intacto por segurança.

Critérios de Pronto (Definition of Done - DoD) para cada Issue

Uma issue só poderá ser movida para a lane **Done - Accepted** no Kanban do GitHub se cumprir os seguintes requisitos:

1. Passar sem falhas na execução do Cucumber local (`cucumber`).
2. Passar sem falhas na execução do RSpec local (`bundle exec rspec`).
3. O histórico da branch conter a evolução atômica dos commits seguindo o padrão `iuricode/padroes-de-commits` .
4. O Pull Request estar revisado pelo par técnico e aprovado, contendo no título o formato exigido: Grupo XX - #NÚMERO_DA_ISSUE Título .
5. A Wiki do repositório estar atualizada documentando a feature e apontando o respectivo responsável.